Metadexer — Technical Specification

Attribute	Value
Project Name	Metadexer
Project Slug	`metadexer`
Domain	metadexer.com (reserved)
Repository	github.com/shruggietech/metadexer
License	Apache License v2.0
Version	Pre-release (0.1.0 target)
Author	William Thompson (ShruggieTech LLC)
Latest Revision Date	2026-03-09
Document Status	DRAFT
Audience	AI-first, Human-second

Table of Contents

- 1. Document Information
  - 1.1. Purpose and Audience
  - 1.2. Scope
  - 1.3. Document Maintenance
  - 1.4. Conventions Used in This Document
  - 1.5. Terminology
  - 1.6. Reference Documents
- 2. Project Overview
  - 2.1. Project Identity
  - 2.2. Relationship to shruggie-indexer
  - 2.3. Design Goals
  - 2.4. Non-Goals
  - 2.5. Platform and Runtime Requirements
- 3. Architecture
  - 3.1. Component Map
  - 3.2. Module Responsibilities
  - 3.3. Data Flow
  - 3.4. Operational Modes
- 4. The IndexEntry Contract
  - 4.1. Ownership and Authority
  - 4.2. Schema Location
  - 4.3. Schema Evolution Rules
  - 4.4. Consumer Obligations

- 5. Vault Module
  - 5.1. Purpose
  - 5.2. Operations
  - 5.3. Storage Backends
  - 5.4. Invariants
  - 5.5. Verification Modes
- 6. Catalog Module
  - 6.1. Purpose
  - 6.2. Operations
  - 6.3. Database Backends
  - 6.4. Hybrid Storage Routing
  - 6.5. Catalog-Indexer Contract
  - 6.6. Invariants
- 7. Sync Module
  - 7.1. Purpose
  - 7.2. Pipeline Stages
  - 7.3. Sync Plans
  - 7.4. Invariants
- 8. Reference and Deletion Model
  - 8.1. Asset Immutability
  - 8.2. Two-Phase Deletion
- 9. Failure Model
  - 9.1. Tolerated Failure Modes
  - 9.2. Recovery Principles
- 10. Repository Structure
  - 10.1. Top-Level Layout
  - 10.2. Source Package Layout
  - 10.3. Documentation Artifacts
  - 10.4. Scripts and Build Tooling
- 11. Documentation Site
  - 11.1. Site Configuration
  - 11.2. Navigation Structure
  - 11.3. Changelog Synchronization
  - 11.4. Build and Preview
  - 11.5. Deployment
  - 11.6. Dependencies
- 12. CLI Interface
  - 12.1. Command Structure
  - 12.2. CLI Conventions
- 13. Configuration
  - 13.1. Configuration Architecture
  - 13.2. Application Data Directory
- 14. File Encoding and JSON Conventions
  - 14.1. File Encoding and Line Endings
  - 14.2. JSON Conventions
- 15. External Dependencies

    - 15.1. Python Dependencies
    - 15.2. External Service Dependencies
    - 15.3. Dependency Verification at Runtime
- 16. Logging and Diagnostics
- 17. Testing
- 18. Packaging and Distribution
- 19. Platform Portability
    - 19.1. Cross-Platform Design Principles
    - 19.2. Platform-Specific Considerations
- 20. Security and Safety
    - 20.1. Path Validation and Sanitization
    - 20.2. Destructive Operation Safeguards
    - 20.3. Credential and Secret Handling
    - 20.4. Large File and Resource Limit Handling
- 21. Performance Considerations
    - 21.1. Ingestion Pipeline Performance
    - 21.2. Catalog Query Performance
    - 21.3. Vault I/O Performance
- 22. Development Phases
    - 22.1. Phase 2: Storage and Catalog
    - 22.2. Phase 3: Pipeline
    - 22.3. Phase 4: Search and Scale
    - 22.4. Phase 5: Integration and Polish
- 23. Development Workflow
    - 23.1. Purpose and Authority
    - 23.2. Workflow Layers
    - 23.3. Handoff Protocol
    - 23.4. Agent Execution Model
    - 23.5. Git Conventions
    - 23.6. Agent Context Management
    - 23.7. Review and Integration
    - 23.8. Tooling Configuration
    - 23.9. Workflow Summary
- 24. Composition Rules
- 25. Future Considerations

---

1. Document Information

1.1. Purpose and Audience

This document is the authoritative technical specification for `metadexer`, a content-addressed asset management system that handles storage, cataloging, deduplication, and search across large, heterogeneous collections of digital data.

The specification is written for an **AI-first, Human-second** audience. Its primary consumers are AI implementation agents operating within isolated context windows during sprint-based development. Every

section provides sufficient detail for an AI agent to produce correct design decisions without requiring interactive clarification. Human developers and maintainers are the secondary audience.

This specification describes:

- The architectural decomposition of metadexer into its three internal modules (vault, catalog, sync) and their relationship to the external `shruggie-indexer` dependency.
- The behavioral contracts, invariants, and responsibility boundaries for each module.
- The repository structure, documentation site, configuration architecture, CLI design, and operational modes.
- The platform portability, security, and performance constraints that govern all implementation work.
- The development phasing that governs implementation ordering.
- The development workflow that governs how planning, implementation, review, and integration are conducted across all development sessions.

This specification does NOT serve as a user guide, tutorial, or API reference. Those artifacts are separate deliverables.

1.2. Scope

In Scope

- **Vault module.** Content-addressed byte storage with local filesystem and S3-compatible backends. Put, get, head, verify, and prune operations.
- **Catalog module.** Metadata registry and search engine. IndexEntry ingestion, field projection, basic and full-text search, hybrid storage routing, reference tracking, and temporal correlation. PostgreSQL and SQLite backends.
- **Sync module.** Ingestion pipeline orchestration. Sync Plan generation, deduplication, storage routing, resumable uploads, and idempotent catalog commits.
- **CLI interface.** The canonical user interface for all operations, built on `click`.
- **Configuration system.** Layered TOML configuration with compiled defaults, user config, project-local config, and CLI overrides.
- **Documentation site.** MkDocs-based documentation with the Material for MkDocs theme, deployed via GitHub Pages.
- **Packaging and distribution.** PyInstaller-based standalone executables distributed via GitHub Releases.
- **Development workflow.** Two-layer workflow model (admin/coding), handoff protocol, agent execution model, agent context management, and review procedures.

Out of Scope

- **shruggie-indexer internals.** The indexer has its own specification (`shruggie-indexer-spec.md`). metadexer consumes the IndexEntry contract; it does not define or extend it.
- **Web UI.** A browser-based interface for search and browsing is a future extension (§25), not part of the core specification.
- **hotwire integration.** Automated real-time feed ingestion via the hotwire pipeline is a future extension.
- **Embedding and vector search.** Semantic retrieval via vector similarity is a future extension layered on top of the catalog.

1.3. Document Maintenance

This specification is maintained as a living document alongside the codebase. When the specification and the implementation disagree, the specification is presumed correct unless a deliberate amendment has been recorded in the document history.

1.4. Conventions Used in This Document

This specification uses the requirement level keywords defined in RFC 2119. These keywords are capitalized when used in their RFC 2119 sense:

Keyword	Meaning
MUST / **MUST NOT**	Absolute requirement or prohibition.
SHALL / **SHALL NOT**	Synonymous with MUST / MUST NOT.
SHOULD / **SHOULD NOT**	Strong recommendation. Deviation must be deliberate.
MAY	Truly optional.

Typographic conventions:

- `Monospace` denotes code identifiers, file paths, CLI flags, configuration keys, and literal values.
- **Bold** denotes emphasis or key terms being defined.
- *Italic* denotes document titles, variable placeholders, or first use of a defined term.
- `§N.N` denotes a cross-reference to a section within this specification.

Code examples use Python syntax unless otherwise noted. Examples are illustrative; they demonstrate intent and structure but are not necessarily the exact implementation.

1.5. Terminology

Term	Definition
IndexEntry	A structured JSON record produced by shruggie-indexer that serves as the authoritative metadata description of a content object. Defined by the v2 schema.
Content-addressed	An identity model where objects are identified by a hash of their byte content, not by filename or path.
Vault	The metadexer module responsible for raw byte storage under deterministic, content-derived keys.
Catalog	The metadexer module responsible for metadata storage, indexing, search, and reference tracking.
Sync	The metadexer module responsible for orchestrating the ingestion pipeline across the indexer, vault, and catalog.
Sync Plan	A dry-run preview of pending operations generated by the sync module before committing any changes.

Term	Definition
Storage routing	The rule-based decision process that determines whether ingested content is stored inline in the catalog database or routed to the vault as binary blob storage.
Sidecar file	An external metadata file that lives alongside the file it describes, identified by filename pattern matching (e.g., `video.mp4` may have a sidecar `video.srt`). Sidecar discovery and parsing is handled by shruggie-indexer.
Reference	A mutable pointer (collection, project, tenant, snapshot) that links to an immutable asset in the catalog.
Prune	The explicit operation that removes unreferenced objects from the vault. Never triggered automatically.
`storage_name`	The deterministic filename derived from an item's `id` and extension, produced by shruggie-indexer. Used as the vault storage key.
Admin layer	The strategic planning tier of the development workflow. Operates in browser-based AI chat sessions. Produces specifications, sprint plans, and agent context files. Does not produce code.
Coding layer	The implementation tier of the development workflow. Operates in IDE-based or terminal-based AI coding agent sessions. Consumes sprint plans and specifications, produces code.
Sprint document	A structured planning artifact produced by the admin layer that defines a batch of work items for the coding layer. Filed in `.handoff/plans/`.
Session report	A structured summary produced by (or on behalf of) the coding layer at the end of a development session. Filed in `.handoff/reports/`.
Agent context file	A repository-root file (`CLAUDE.md`, `.github/copilot-instructions.md`) that provides persistent project-level context to AI coding agents.

1.6. Reference Documents

Document	Location	Description
metadexer Overview	`.archive/20260305-004-metadexer-overview.md`	High-level project overview covering vision, architecture, use cases, and development roadmap. This specification supersedes the overview for all technical details.
shruggie-indexer Spec	`shruggie-indexer` repository: `shruggie-indexer-spec.md`	The authoritative technical specification for the indexer. Defines the IndexEntry v2 schema, all indexing behavior, sidecar handling, and output contracts.
IndexEntry v2 Schema	schemas.shruggie.tech/data/shruggie-indexer-v2.schema.json	The canonical JSON Schema definition for the v2 IndexEntry format.

Document	Location	Description
Sprint Document Format	metadexer Overview, Appendix D	Defines the five-section structure for sprint planning documents: header block, purpose and context, implementation ordering, work item sections, and specification update directive.

2. Project Overview

2.1. Project Identity

Property	Value
Product name	metadexer
Organization	ShruggieTech LLC (https://shruggie.tech)
Author	William Thompson
Language	Python 3.12+
Package name	`metadexer`
CLI command	`metadexer`
Domain	metadexer.com (reserved)

metadexer is the product name. It is what users install, what the CLI is called, what the documentation refers to, and what appears on metadexer.com.

ShruggieTech is the organizational identity (ShruggieTech LLC). It is the publisher of both shruggie-indexer and metadexer. It appears in license headers, copyright notices, and GitHub organization naming. It is not a product name.

2.2. Relationship to shruggie-indexer

shruggie-indexer is a standalone tool that predates the metadexer product identity. It has its own repository, its own release schedule, its own specification, and its own branding. It is a dependency of metadexer, not a sub-component of it.

The relationship is defined by a single contract: the **IndexEntry v2 JSON schema**. shruggie-indexer produces IndexEntry records; metadexer consumes them. metadexer MUST NOT redefine IndexEntry fields or semantics. metadexer MUST NOT recompute content identity. The indexer defines truth; metadexer preserves, records, and propagates it.

shruggie-indexer v0.1.2 is the current stable release at the time of this writing. It defines identity, extracts metadata, manages sidecars, and produces well-formed IndexEntry v2 JSON.

2.3. Design Goals

The following properties are preserved across all implementation decisions:

- **G1 - Determinism.** The same input always produces the same output.
- **G2 - Idempotence.** Repeating an operation produces no additional side effects.
- **G3 - Composability.** The indexer works independently; metadexer modules work as a coordinated unit.
- **G4 - Auditability.** All operations are traceable; no silent mutations or deletions.
- **G5 - Minimal hidden state.** System state is observable and recoverable from durable records.
- **G6 - Clear failure modes.** Failures surface explicitly; the system does not paper over errors.
- **G7 - Offline-first capability.** Local operation is fully functional without network access.
- **G8 - CLI-first design.** CLI contracts are the canonical interface. API and GUI layers are secondary and thin.
- **G9 - Cross-platform operation.** The application MUST run on Windows, Linux, and macOS from a single codebase.

2.4. Non-Goals

metadexer is explicitly not:

- A traditional Digital Asset Management system optimized for marketing workflows and approval pipelines.
- A web-first application that requires a browser for core functionality.
- A SaaS product with per-seat licensing.
- A general-purpose object storage layer (MinIO, raw S3 already do this).
- A tool that makes autonomous decisions about user data.

2.5. Platform and Runtime Requirements

Requirement	Value
Python version	3.12+ (enables `tomllib`, modern type hints)
Target platforms	Windows (x64), Linux (x64), macOS (arm64)
External binary dependencies	`exiftool` (required by shruggie-indexer, not directly by metadexer)
Configuration format	TOML (parsed by `tomllib`)
JSON serializer	`orjson` preferred, `json` stdlib as silent fallback

3. Architecture

3.1. Component Map

```
shruggie-indexer (standalone tool, own repository)
    │
    │  produces IndexEntry v2 JSON
    │
    ▼
metadexer (single application, single repository)
    ├── vault module    - content-addressed byte storage
    ├── catalog module  - metadata registry, search, references
    └── sync module     - ingestion pipeline orchestration
```

The indexer is a standalone tool because it genuinely is one. It operates on local files, produces JSON output, has no dependency on metadexer, and is useful on its own. People use it independently to index files, generate metadata, and pipe the output into their own scripts.

The vault, catalog, and sync modules share the IndexEntry contract, share configuration, share a data lifecycle, and are invoked together in normal operation. They live in a single repository as internal modules of the metadexer application. Each module maintains a clean responsibility boundary (the vault module does not know about catalog references; the catalog module does not store raw bytes), but they are not separate projects.

3.2. Module Responsibilities

Concern	Owner
Identity generation and metadata extraction	shruggie-indexer
Raw byte storage	metadexer vault module
Structured metadata, references, and search	metadexer catalog module
Pipeline orchestration	metadexer sync module

No module is permitted to silently absorb a neighbor's responsibility.

3.3. Data Flow

The standard ingestion pipeline follows this sequence:

1. The sync module accepts a directory or file target as input.
2. The sync module invokes shruggie-indexer to produce IndexEntry records for each target.
3. For each IndexEntry, the sync module checks the catalog and vault for already-present assets. Duplicates are skipped.
4. Storage routing rules examine the IndexEntry. Small text-based content is routed to inline catalog storage. Large or binary content is routed to the vault.
5. Bytes are uploaded to the vault (for vault-routed content).
6. The IndexEntry is committed to the catalog only after upload is confirmed complete.

3.4. Operational Modes

Local mode. Indexer, local vault, local catalog. No server required. Suitable for offline archival and single-user workflows.

Remote mode. The client invokes the indexer and computes identity locally. The client uploads bytes to a remote vault. The client commits metadata to a remote catalog. The server MAY verify integrity asynchronously by policy.

In both modes, identity always originates from the client. The sync module never delegates identity decisions to a server.

---

4. The IndexEntry Contract

4.1. Ownership and Authority

The IndexEntry v2 JSON schema is defined and owned by shruggie-indexer. metadexer consumes it as a fixed contract. The authoritative behavioral specification for IndexEntry fields, types, and semantics lives in the shruggie-indexer specification (`shruggie-indexer-spec.md`, §5). This document does not duplicate that content.

metadexer MUST NOT redefine IndexEntry fields or their semantics. metadexer MUST NOT add fields to, remove fields from, or structurally alter an IndexEntry record after it is produced by the indexer. An IndexEntry received by metadexer is treated as an immutable artifact.

4.2. Schema Location

The authoritative machine-readable schema is hosted at:

```
https://schemas.shruggie.tech/data/shruggie-indexer-v2.schema.json
```

This document uses JSON Schema Draft-07. A local copy is committed to the indexer repository at `docs/schema/shruggie-indexer-v2.schema.json`.

4.3. Schema Evolution Rules

These rules are owned by the shruggie-indexer specification and summarized here for metadexer implementers:

- **Additive fields are non-breaking.** New optional fields MAY be added to v2 without incrementing `schema_version`. Consumers MUST tolerate unknown optional fields.
- **Structural changes require a version bump.** Renaming, retyping, removing a required field, or altering semantic meaning constitutes a breaking change and MUST increment `schema_version`.
- **Deprecation before removal.** A field marked deprecated in version N is emitted but ignored, then removed in version N+1.
- **Consumers dispatch on `schema_version`.** The integer value `2` is checked before parsing. Documents with unrecognized versions SHOULD be rejected.

4.4. Consumer Obligations

metadexer, as an IndexEntry consumer, MUST:

- Validate `schema_version` before processing.
- Tolerate unknown optional fields without error.
- Reject records with unrecognized `schema_version` values.
- Never recompute identity fields (`id`, `storage_name`, `hashes`) unless performing an explicit integrity verification.
- Never rewrite or amend an IndexEntry outside of the indexer's own operation.

---

5. Vault Module

5.1. Purpose

The vault module provides content-addressed byte storage. It preserves raw file content under deterministic keys derived from the IndexEntry's `storage_name` field. The vault is responsible for bytes, not metadata.

5.2. Operations

Operation	Description
put	Store bytes under the `storage_name` key. Write-once: if the key already exists with identical content, the operation is a no-op. If the key exists with different content, this is a hash collision and MUST be surfaced as an error.
get	Retrieve bytes by `id` or `storage_name`.
head	Check existence of an object without retrieving its bytes.
verify	Re-hash stored bytes and compare against a provided IndexEntry. Verification is always explicit, never triggered automatically during ingest.
prune	Remove unreferenced objects. Pruning is always explicit, never triggered automatically. Requires reconciliation against the catalog to determine which objects are unreferenced.

5.3. Storage Backends

Local filesystem. The primary backend for local-mode operation. Objects are stored as files in a content-addressed directory structure under a configurable vault root path.

S3-compatible object storage. For remote or hybrid deployments. Compatible with AWS S3, MinIO, MEGA S4, and any S3-compatible API. Objects are stored as keys in a configured bucket using the `storage_name` as the object key.

The local backend is the first implementation target. The S3 backend is the second.

5.4. Invariants

- The same `storage_name` always maps to identical bytes (write-once guarantee).
- The vault enforces identity; it does not compute it.
- Verification is always explicit, never triggered automatically during ingest.
- Pruning is always explicit. The vault never autonomously deletes content.
- The vault module has no knowledge of catalog references, collections, or search. It stores and retrieves bytes.

5.5. Verification Modes

Mode	Description
Strict	Re-hash the entire stored object and compare against the IndexEntry.
Sampled	Probabilistic audits across a fraction of stored objects.

Mode	Description
Tiered	Frequency and depth determined by policy or asset risk classification.

Verification is invoked by policy or on-demand. It is not part of the ingest critical path. This preserves ingestion performance while maintaining the ability to audit integrity continuously.

---

6. Catalog Module

6.1. Purpose

The catalog module is the metadata registry and search engine. It records and indexes everything metadexer knows about every object. It stores IndexEntry records, projects searchable fields, tracks logical references, provides query capability, and maintains the temporal history of observed objects.

6.2. Operations

Operation	Description
ingest	Accept an IndexEntry record and store it. Project and index searchable fields. Duplicate ingest (same `id`) is idempotent.
search	Query indexed fields including MIME type, size, timestamps, name, extension, and (for string-stored content) full-text search over content bodies.
reference	Create, list, and remove logical references (collections, projects, tenants, snapshots) that point to cataloged assets.
correlate	Link IndexEntry snapshots across time using `id`, `session_id`, and `indexed_at` to build identity evolution history.
reconcile	Compare catalog contents against the vault to detect missing or orphaned blobs.

6.3. Database Backends

PostgreSQL is the primary backend. It is built for power users running serious workloads: millions of objects, full-text search across years of ingested data, persistent infrastructure on a home server or dedicated machine. It is the default for anyone who takes their data seriously.

SQLite is a lightweight alternative for quick evaluation, portable single-file deployments, and casual use. It is fully functional but not optimized for the concurrent access patterns or query complexity that arise at scale.

6.4. Hybrid Storage Routing

The catalog implements hybrid storage routing. Small text-based content (e.g., a 500-byte JSON response from an API feed) is stored inline as string data within the catalog database for instant full-text search. Large or binary content (e.g., a 4 GB video file) is routed to the vault. Both types are tracked by the same catalog with the same identity model.

Storage routing is determined by configurable rulesets and by explicit user direction at ingestion time. The rulesets consider factors including file size, MIME type, and content characteristics. The exact threshold and

rule configuration is defined in the configuration system (§13).

6.5. Catalog-Indexer Contract

The IndexEntry is a point-in-time snapshot. Its fields describe a file's identity, metadata, and filesystem state at the moment of indexing. Over time, content hashes change when content is modified, timestamps shift through normal filesystem operations, metadata evolves as external tools and source files change, and relative paths change when files are moved or index roots differ between runs.

This transient nature is correct by design. The indexer produces accurate snapshots. The catalog receives them, correlates them across time, and maintains a durable record of identity evolution.

6.6. Invariants

- `id` is globally unique per content object. Duplicate ingest is idempotent.
- Multiple references MAY point to a single asset. Assets do not belong to references.
- Reference deletion removes the reference, not the underlying asset.
- Physical deletion from the vault requires a separate, explicit prune operation.
- The catalog does not store raw bytes (except inline string content per hybrid routing rules).

---

7. Sync Module

7.1. Purpose

The sync module is the ingestion pipeline orchestrator. It connects the indexer, vault, and catalog into a reliable, resumable workflow. It is the primary interface through which data enters the metadexer system.

7.2. Pipeline Stages

1. Accept directory or file targets as input.
2. Invoke shruggie-indexer to produce IndexEntry records for each target.
3. Check the catalog and vault for already-present assets to avoid redundant work.
4. Apply storage routing rules to determine destination (vault or inline catalog storage).
5. Upload bytes to the vault for vault-routed content.
6. Commit IndexEntry records to the catalog only after upload is confirmed complete.

The sync module supports dry-run mode, in which it executes stages 1 through 4 and produces a Sync Plan (§7.3) without committing any changes.

7.3. Sync Plans

A Sync Plan is a dry-run preview of pending operations. It is generated before any data is written and describes exactly what the sync module intends to do: which objects are new, which are duplicates that will be skipped, which will be routed to the vault, and which will be stored inline in the catalog. The Sync Plan is the user's opportunity to review and approve before committing.

7.4. Invariants

- Identity always originates from the client. Sync never delegates identity decisions to a server.

- Upload is confirmed complete before catalog commit.
- Catalog commit is idempotent. Resubmitting the same IndexEntry is safe.
- Sync is restartable at any point without risk of corruption or data loss.
- No asset is silently overwritten or deleted.
- Resumable operation across interrupted runs. An interrupted sync resumes from the point of interruption, not from the beginning.

---

8. Reference and Deletion Model

8.1. Asset Immutability

Assets are **immutable**. Once bytes are stored in the vault and an IndexEntry is committed to the catalog, the content object is fixed. References are **mutable**. Collections, projects, tenants, and snapshots are pointers that can be created, updated, and removed without affecting the underlying asset.

8.2. Two-Phase Deletion

Deletion is always two-phase:

1. Remove references in the catalog.
2. Explicitly invoke vault prune to remove unreferenced objects.

This model prevents accidental data loss. No content is ever removed in a single implicit operation. Both phases require explicit user action.

---

9. Failure Model

9.1. Tolerated Failure Modes

metadexer MUST tolerate and recover from:

- Interrupted or partial uploads.
- Duplicate catalog commits.
- Partial batch failures (some objects in a batch succeed, others fail).
- Network failures at any stage of remote-mode operation.
- Temporary unavailability of either the catalog database or vault storage.

9.2. Recovery Principles

- **Consistency over convenience.** Incomplete operations leave the system in a known, recoverable state.
- **Explicit reconciliation over implicit repair.** The system surfaces discrepancies and requires deliberate action to resolve them.

---

10. Repository Structure

10.1. Top-Level Layout

```
metadexer/
├── .archive/
├── .github/
│   ├── copilot-instructions.md
│   └── workflows/
├── .handoff/
│   ├── plans/
│   └── reports/
├── docs/
├── scripts/
├── src/metadexer/
├── tests/
├── .gitignore
├── .python-version
├── CHANGELOG.md
├── CLAUDE.md
├── LICENSE
├── mkdocs.yml
├── pyproject.toml
├── README.md
└── metadexer-spec.md
```

Path	Type	Description
`.archive/`	Directory	Historical storage for completed, superseded, or retired project documents. Not part of the active handoff flow. Documents that were once active in `.handoff/plans/` MAY be moved here after their associated sprint is complete and integrated. Files follow the standard ShruggieTech naming convention: `<YYYYmmdd>-<ZZZ>-<title>.<ext>`, where `ZZZ` is a three-digit zero-padded increment that resets to `001` on each new date.
`.github/`	Directory	GitHub-specific repository configuration. Contains `copilot-instructions.md` (project-level AI coding guidelines for GitHub Copilot) and `workflows/` with CI/CD pipeline definitions: `release.yml` for the release build pipeline (§18) and `docs.yml` for automated documentation site deployment to GitHub Pages (§11).
`.handoff/`	Directory	Bidirectional handoff location for artifacts exchanged between the admin layer and coding layer. See §23.3 for the full handoff protocol.
`.handoff/plans/`	Directory	Admin-to-coding handoff. Contains sprint planning documents, session prompt templates (`_TEMPLATE.txt`), and any other artifacts the admin layer produces for consumption by the coding layer. Files follow the same date-scoped naming convention as `.archive/`.
`.handoff/reports/`	Directory	Coding-to-admin handoff. Contains session reports, test result summaries, and other artifacts produced by (or on behalf of) the coding layer for consumption by the admin layer. See §23.3.4 for the required report format.

Path	Type	Description
`docs/`	Directory	All project documentation for the MkDocs-based documentation site. See §10.3.
`scripts/`	Directory	Platform-paired shell scripts for development environment setup and build automation. See §10.4.
`src/metadexer/`	Directory	The Python source package. All importable code lives here. See §10.2.
`tests/`	Directory	All test code. Organized by test type, not by source module. See §17.
`.gitignore`	File	Standard Python `.gitignore` covering `__pycache__/`, `*.pyc`, `.venv/`, `dist/`, `build/`, `*.egg-info/`, `site/`, IDE/editor files, and OS artifacts.
`.python-version`	File	Contains the string `3.12` (no minor patch). Used by `pyenv` and similar version managers to auto-select the correct interpreter.
`CHANGELOG.md`	File	Project changelog following Keep a Changelog format. Documents all notable changes organized by release version.
`CLAUDE.md`	File	Project-level agent context file for Claude Code. Read automatically by Claude Code at session start. See §23.6.1 for required contents.
`LICENSE`	File	License text (license selection TBD).
`mkdocs.yml`	File	MkDocs configuration for the documentation site. See §11.
`pyproject.toml`	File	Centralized project metadata, build system configuration, dependency declarations, entry points, and tool settings (`ruff`, `pytest`).
`README.md`	File	Project overview, installation instructions, quick-start usage examples, and links to full documentation.
`metadexer-spec.md`	File	This technical specification. Lives at the repository root for top-level visibility.

10.2. Source Package Layout

```
src/metadexer/
├── __init__.py
├── _version.py
├── cli.py
├── vault/
│   ├── __init__.py
│   ├── store.py              # core put/get/head/verify logic
│   └── backends/
│       ├── local.py          # local filesystem backend
│       └── s3.py             # S3-compatible backend
├── catalog/
│   ├── __init__.py
│   ├── ingest.py             # IndexEntry ingestion
```

```
│   ├── search.py            # query interface
│   └── backends/
│       ├── sqlite.py
│       └── postgres.py
└── sync/
    ├── __init__.py
    ├── pipeline.py          # orchestration logic
    └── plan.py              # Sync Plan generation
```

10.3. Documentation Artifacts

```
docs/
├── index.md
├── changelog.md
├── assets/
│   └── images/
├── getting-started/
│   ├── installation.md
│   └── quickstart.md
└── user-guide/
    ├── index.md
    ├── cli-reference.md
    ├── configuration.md
    └── platform-notes.md
```

Path	Purpose
`index.md`	Documentation site landing page. Provides a project overview, quick-links section, and navigational entry points. Rendered as the site home page by MkDocs.
`changelog.md`	Auto-copied from `CHANGELOG.md` at the repository root. Contains a header comment identifying it as a generated file. See §11.3.
`assets/images/`	Static image assets used by the documentation site (social previews, screenshots).
`getting-started/`	Onboarding documentation: installation guide and quick-start tutorial.
`user-guide/`	End-user documentation: CLI reference, configuration reference, and platform notes. Pages are populated incrementally as features stabilize.

This layout is intentionally minimal at the DRAFT stage. Additional sections (e.g., schema reference, API documentation) are added as the corresponding features are implemented.

10.4. Scripts and Build Tooling

```
scripts/
├── venv-setup.sh            # Linux/macOS: create venv, install deps
├── venv-setup.ps1           # Windows: create venv, install deps
└── build.sh                 # PyInstaller build script (CI and local)
```

Script	Purpose
`venv-setup.sh` / `venv-setup.ps1`	Create a Python virtual environment and install development dependencies from `pyproject.toml`. These scripts are the documented entry point for new developer setup.
`build.sh`	Invokes PyInstaller to produce standalone executables. Used by the GitHub Actions release pipeline and available for local builds.

Scripts MUST be idempotent: running them multiple times produces the same result without error.

---

11. Documentation Site

The project documentation is published as a static site built with MkDocs using the Material for MkDocs theme. This matches the toolchain established by shruggie-indexer.

11.1. Site Configuration

The site is configured by `mkdocs.yml` at the repository root. Key configuration settings:

Setting	Value	Purpose
`site_name`	`metadexer`	Displayed in the site header and browser title.
`site_description`	Project tagline for SEO and social metadata.	
`site_url`	The GitHub Pages URL for the project.	Base URL for canonical links and sitemap generation.
`docs_dir`	`docs`	MkDocs reads all documentation source from the `docs/` directory.
`theme.name`	`material`	Activates the Material for MkDocs theme.
`theme.palette.scheme`	`slate`	Dark mode enabled by default.
`theme.features`	Navigation tabs, instant loading, search highlighting, content tabs.	Provides a polished, responsive documentation experience.

Required Markdown extensions: `admonition`, `pymdownx.details`, `pymdownx.superfences`.

The `nav` key in `mkdocs.yml` defines the sidebar navigation structure explicitly rather than relying on directory auto-discovery. This ensures predictable ordering and human-readable section labels.

11.2. Navigation Structure

```yaml
nav:
  - Home: index.md
  - Getting Started:
      - Installation: getting-started/installation.md
      - Quick Start: getting-started/quickstart.md
  - User Guide:
      - Overview: user-guide/index.md
      - CLI Reference: user-guide/cli-reference.md
      - Configuration: user-guide/configuration.md
      - Platform Notes: user-guide/platform-notes.md
  - Changelog: changelog.md
```

This navigation structure is intentionally minimal at the DRAFT stage. Additional sections are added as features are implemented.

11.3. Changelog Synchronization

The documentation site's changelog page (`docs/changelog.md`) is a copy of `CHANGELOG.md` at the repository root. The copy is maintained manually. The file begins with a header comment identifying it as auto-copied:

```
<!-- THIS FILE IS AUTO-COPIED FROM CHANGELOG.md AT THE REPOSITORY ROOT. -->
<!-- DO NOT EDIT THIS FILE DIRECTLY. Edit CHANGELOG.md instead. -->
```

The canonical changelog is `CHANGELOG.md` at the repository root. All edits are made there. When the documentation site is built or deployed, `docs/changelog.md` MUST reflect the current state of `CHANGELOG.md`. If the two files diverge, the root `CHANGELOG.md` is authoritative.

11.4. Build and Preview

Command	Purpose
`mkdocs serve`	Starts a local development server with live reload at `http://127.0.0.1:8000/`.
`mkdocs build`	Produces the static site in the `site/` directory. The `site/` directory is listed in `.gitignore` and is never committed.

11.5. Deployment

The documentation site is deployed to GitHub Pages via a dedicated GitHub Actions workflow (`.github/workflows/docs.yml`). The workflow:

- **Triggers** on push to `main` when files in `docs/` or `mkdocs.yml` change.
- **Builds** the site using `mkdocs build --strict` (strict mode fails the build on warnings such as broken links or missing pages).
- **Deploys** the built `site/` directory to the `gh-pages` branch using `mkdocs gh-deploy --force`.

The `--strict` flag ensures that documentation quality is enforced in CI. Broken internal links, missing navigation targets, and unreferenced pages cause build failures rather than silent degradation.

11.6. Dependencies

`mkdocs` and `mkdocs-material` are added as optional development dependencies in `pyproject.toml` under a `[project.optional-dependencies]` docs group:

```
[project.optional-dependencies]
docs = [
    "mkdocs>=1.6",
    "mkdocs-material>=9.5",
]
```

These packages are NOT required for using, developing, or testing metadexer. They are required only for building or previewing the documentation site. Documentation authors install them explicitly with `pip install -e ".[docs]"`.

---

12. CLI Interface

12.1. Command Structure

The CLI uses `click` as the argument parser. The top-level command is `metadexer`. Subcommands map to module operations. The exact subcommand tree is defined during implementation, but the following structure reflects the expected shape:

```
metadexer ingest <target>       # run the sync pipeline on a target directory or
file
metadexer search <query>        # query the catalog
metadexer vault verify [...]    # explicit vault verification
metadexer vault prune [...]     # explicit vault pruning
metadexer catalog reconcile     # reconcile catalog against vault
metadexer config [...]          # configuration management
```

12.2. CLI Conventions

- Content filtering flags are independent of output destination flags.
- `stdout` stays clean for structured output (JSON). All diagnostics go to `stderr`.
- Destructive operations require explicit opt-in flags.
- `--dry-run` mode is available for any operation with side effects.
- CLI contracts are the canonical interface. API and daemon layers are secondary and MUST NOT implement independent behavior.

---

13. Configuration

13.1. Configuration Architecture

All configuration uses **TOML** format, parsed by Python's `tomllib` module.

Layered override behavior (lowest to highest priority):

1. Compiled defaults (always present; a Python module, not a TOML file).
2. User config directory (platform-specific; see §13.2).
3. Project-local config (searched upward from target directory).
4. CLI arguments (highest priority).

Configuration objects SHOULD be frozen (immutable) dataclasses. Unknown keys in user-provided TOML MUST be silently ignored (forward compatibility).

13.2. Application Data Directory

Platform	Path
Windows	`%LOCALAPPDATA%\metadexer\`
Linux	`$XDG_CONFIG_HOME/metadexer/` (default: `~/.config/metadexer/`)
macOS	`~/Library/Application Support/metadexer/`

---

14. File Encoding and JSON Conventions

14.1. File Encoding and Line Endings

All text files in the repository use UTF-8 encoding without BOM and LF (`\n`) line endings. `.gitattributes` enforces `* text=auto eol=lf` to normalize line endings on commit.

14.2. JSON Conventions

- All JSON output uses 2-space indentation by default. Compact (no-whitespace) output is available via CLI flag.
- Keys are `snake_case`.
- `null` is used (not omitted) for explicitly absent values on required fields. Optional fields that are `None` are omitted from output.
- Non-ASCII characters are preserved as literal UTF-8, not escaped to `\uXXXX` sequences.
- `orjson` is preferred for performance where available, with `json.dumps()` as a silent fallback.

---

15. External Dependencies

15.1. Python Dependencies

The full dependency inventory is defined during implementation in `pyproject.toml`. The following are known required dependencies:

Package	Purpose	Category
`click`	CLI argument parsing	Required
`orjson`	High-performance JSON serialization	Optional (silent fallback to `json` stdlib)

Package	Purpose	Category
`psycopg` (or equivalent)	PostgreSQL database driver	Required for PostgreSQL backend
`boto3` (or equivalent)	S3-compatible object storage client	Required for S3 backend
`shruggie-indexer`	IndexEntry production	Required (invoked as subprocess or library)

15.2. External Service Dependencies

Service	Required	Notes
PostgreSQL	For PostgreSQL catalog backend	Not required when using SQLite backend
S3-compatible storage	For S3 vault backend	Not required when using local filesystem backend

15.3. Dependency Verification at Runtime

Category	Failure mode
Required CLI dependency (e.g., `click`)	Hard error with install instructions.
Required backend dependency (e.g., `psycopg`)	Hard error when that backend is selected.
Optional performance dependency (e.g., `orjson`)	Silent fallback to stdlib equivalent.
Development/test dependency (e.g., `pytest`)	Import error at test time only.

16. Logging and Diagnostics

All logging uses Python's standard `logging` module. Logger names follow the package structure (e.g., `metadexer.vault.store`, `metadexer.catalog.ingest`, `metadexer.sync.pipeline`).

Log output goes to `stderr` (console) and optionally to persistent log files under `<app_data_dir>/logs/`. Log files use the naming pattern `YYYY-MM-DD_HHMMSS.log`.

17. Testing

Test suites are organized by test type, not by source module:

Category	Directory	Scope
Unit	`tests/unit/`	Individual functions and methods in isolation.
Integration	`tests/integration/`	Full pipeline, end-to-end.
Conformance	`tests/conformance/`	Output structure against canonical schemas and contracts.

Category	Directory	Scope
Platform	`tests/platform/`	OS-specific behavior.

All tests run with a bare `pytest` invocation. `pyproject.toml` registers custom markers with `--strict-markers`. Platform-specific tests use pytest markers (`@pytest.mark.platform_windows`, `@pytest.mark.platform_linux`, `@pytest.mark.platform_macos`) and are executed on the corresponding platform in CI.

---

18. Packaging and Distribution

metadexer is **not published to PyPI**. End users download pre-built executables from GitHub Releases.

Packaging stack:

- `pyproject.toml` as the single metadata and dependency declaration file.
- PyInstaller for standalone executables.
- GitHub Actions release pipeline triggered on `v*` tag pushes, with matrix builds for Windows (x64), Linux (x64), and macOS (arm64).

Release pipeline stages: Checkout, Test, Build (PyInstaller), Rename artifacts (version + platform tags), Upload, Create GitHub Release.

The version string lives in a single `_version.py` file. All other references derive from it. Versioning follows semantic versioning (`MAJOR.MINOR.PATCH`). Pre-release versions use suffixes like `-rc1`.

---

19. Platform Portability

19.1. Cross-Platform Design Principles

Design goal G9 (§2.3) states: the application MUST run on Windows, Linux, and macOS from a single codebase. The following principles govern the approach to platform portability:

No platform-conditional logic in the core modules. The `vault/`, `catalog/`, and `sync/` subpackages MUST NOT contain `if sys.platform == ...` or `if os.name == ...` branches. All platform variation is absorbed by Python standard library abstractions (`pathlib`, `os`, `hashlib`, `subprocess`) or by the configuration system. The one permitted exception is the `cli.py` entry point, which MAY contain platform-conditional logic for presentation-layer concerns (console encoding, terminal setup) that do not affect application behavior.

Forward-slash normalization in stored paths. All path strings stored in the catalog use forward-slash (`/`) separators, regardless of the host platform. This ensures that catalog data is portable across operating systems.

Graceful degradation for unavailable features. When a platform does not support a feature, the application uses the best available approximation and logs a debug-level message on first occurrence. No per-file warnings are produced for known platform limitations.

19.2. Platform-Specific Considerations

Windows. The application data directory resolves under `%LOCALAPPDATA%` (Local), not `%APPDATA%` (Roaming). Long path support (`\\?\` prefix) is handled by Python's `pathlib` where applicable. Console output encoding is set to UTF-8 explicitly.

Linux. The application data directory follows the XDG Base Directory Specification, defaulting to `~/.config/` when `XDG_CONFIG_HOME` is not set.

macOS. The application data directory resolves to `~/Library/Application Support/`. The arm64 architecture is the primary macOS build target.

Platform-specific behaviors are verified by the `tests/platform/` test suite (§17).

---

20. Security and Safety

20.1. Path Validation and Sanitization

All user-supplied path inputs MUST be resolved to canonical absolute paths before use. The application MUST NOT follow symbolic links into directories outside the user-specified target scope unless explicitly directed. Path traversal attacks (e.g., `../../etc/passwd` embedded in filenames or metadata) MUST be detected and rejected.

Vault storage keys are derived from `storage_name` values produced by the indexer (deterministic, content-derived, alphanumeric with a single dot separator). The vault backend MUST validate that storage keys conform to the expected pattern before performing any filesystem or object storage operation.

20.2. Destructive Operation Safeguards

All destructive operations require explicit opt-in flags. This applies to vault prune, reference deletion, catalog entry removal, and any operation that modifies or removes stored data. No destructive action is ever a default. `--dry-run` mode is available for any operation with side effects, and `--dry-run` MUST be the default behavior for prune operations unless an explicit confirmation flag is provided.

20.3. Credential and Secret Handling

Database connection strings, S3 access keys, and other credentials MUST NOT be stored in plaintext configuration files. Credentials are supplied via environment variables or platform-native credential storage mechanisms. Configuration files MAY reference environment variable names but MUST NOT contain credential values directly. Credentials MUST NOT appear in log output at any log level.

20.4. Large File and Resource Limit Handling

The vault module handles files of arbitrary size. File content MUST be read and written in chunks (not loaded entirely into memory) to prevent memory exhaustion on large objects. Hash computation during verification MUST use streaming (chunked) reads. The catalog module MUST handle IndexEntry records of arbitrary depth (deeply nested directory trees with large `items` arrays) without stack overflow. Configurable limits on batch sizes and concurrent operations prevent resource exhaustion during bulk ingestion.

---

21. Performance Considerations

21.1. Ingestion Pipeline Performance

The ingestion pipeline is designed for throughput at scale (millions of files, terabytes of data). Deduplication checks against the catalog and vault MUST be performed before any byte transfer to avoid redundant I/O. Batch operations (catalog commits, vault existence checks) SHOULD be used where the backend supports them to minimize round-trip overhead.

21.2. Catalog Query Performance

The catalog schema MUST include appropriate indexes on commonly queried fields: `id`, `mime_type`, `size.bytes`, `timestamps.modified`, `name.text`, and `extension`. Full-text search over string-stored content uses the database engine's native full-text search capabilities (PostgreSQL `tsvector`/`tsquery`, SQLite FTS5). Index design is documented as part of the catalog backend implementation.

21.3. Vault I/O Performance

Vault operations use chunked I/O for all file reads and writes. The chunk size is configurable with a sensible default (e.g., 8 MB). For the S3 backend, multipart upload is used for objects exceeding the S3 single-upload size threshold. The local filesystem backend uses a directory sharding strategy (e.g., first N characters of `storage_name` as subdirectory prefixes) to prevent performance degradation from directories with millions of entries.

---

22. Development Phases

Phase 1 (Foundation) is complete: shruggie-indexer v0.1.2 is released and stable. The following phases define the remaining implementation ordering.

22.1. Phase 2: Storage and Catalog

Target: A working local-mode system where `metadexer ingest <directory>` indexes files, stores bytes in a local vault, and commits metadata to a catalog with basic query support.

Scope:

- Vault module: put, get, head, verify. Local filesystem backend first. S3-compatible backend second.
- Catalog module: IndexEntry ingest, field projection, basic search (MIME type, size, timestamps, name, extension). PostgreSQL backend as primary target. SQLite backend for portable/evaluation use. Hybrid string storage for small text-based content.

Deferred from this phase: reference tracking (collections/projects/users), temporal correlation, reconciliation, pruning.

22.2. Phase 3: Pipeline

Target: A reliable, resumable ingestion pipeline with dry-run support.

Scope:

- Sync Plan generation (dry-run preview of pending operations).
- Deduplication checks against existing catalog/vault state.

- Storage routing based on configurable rulesets.
- Resumable operation across interrupted runs.
- Idempotent catalog commits.

22.3. Phase 4: Search and Scale

Target: Full catalog capabilities as specified in §6.

Scope:

- Full-text search over string-stored content.
- Reference tracking (collections, projects, tenants).
- Temporal correlation of IndexEntry snapshots.
- Vault reconciliation (detect orphaned or missing blobs).
- Vault pruning for unreferenced objects.

22.4. Phase 5: Integration and Polish

Scope:

- hotwire integration (automated feed ingestion pipeline).
- Web UI for search and browsing (thin layer over API).
- MEGA S4 as a vault backend.
- Documentation site, public release, metadexer.com.

---

23. Development Workflow

23.1. Purpose and Authority

This section defines the standardized development workflow for the metadexer project. It governs how administrative planning, coding, review, and integration are conducted across all development sessions. All participants (human and AI agents) MUST follow this workflow unless an explicit, documented exception is approved by the project lead.

This workflow supersedes all ad-hoc development patterns previously used on the project. It incorporates parallel agent execution, worktree-based isolation, and a structured handoff protocol between strategic planning and implementation.

23.2. Workflow Layers

Development activity is organized into two distinct layers. These layers operate independently and communicate through version-controlled artifacts only. No direct, informal channel (verbal, chat, or otherwise) substitutes for the written handoff artifacts described in §23.3.

23.2.1. Admin Layer

The admin layer is responsible for strategic direction, specification authoring, sprint planning, brainstorming, and roadblock resolution. It operates in browser-based AI chat sessions (Claude.ai projects, or equivalent) where rich project history and personal context are available.

Outputs produced by the admin layer:

Artifact	Description	Destination
Technical specifications	`metadexer-spec.md`, `shruggie-indexer-spec.md`. The authoritative source of truth for all architectural and behavioral decisions.	Repository root
Sprint planning documents	Structured work item definitions following the five-section format from the metadexer overview (Appendix D): header block, purpose and context, implementation ordering, work item sections, and specification update directive.	`.handoff/plans/`
Session prompt templates	`_TEMPLATE.txt` files paired with sprint documents. Contain the initial prompt text to be fed to coding agents at session start.	`.handoff/plans/` (same increment as paired sprint doc)
Agent context files	`CLAUDE.md` and `.github/copilot-instructions.md`. Persistent project-level context consumed by AI coding agents. See §23.6.1.	Repository root and `.github/` respectively
Workflow updates	Updates to this specification's §23 section when process changes are needed.	Repository root (within `metadexer-spec.md`)

The admin layer MUST NOT make direct code changes. All implementation flows through the coding layer.

23.2.2. Coding Layer

The coding layer is responsible for implementing the work defined by the admin layer. It operates in IDE-based and terminal-based AI coding agent sessions (Claude Code, VS Code background/cloud agents, or equivalent tools).

Inputs consumed by the coding layer:

Artifact	Source	How it is obtained
Sprint planning document	`.handoff/plans/`	The project lead provides the file path or pastes the content into the agent's context window. The agent reads it from the repository if operating in a worktree with the latest `main` state.
Session prompt template	`.handoff/plans/`	Fed to the agent as the initial prompt at session start.

Artifact	Source	How it is obtained
Technical specification	Repository root	The agent reads the specification file for the component under modification. The project lead MAY explicitly include it in the context window or instruct the agent to read it.
Agent context file	`CLAUDE.md` (auto-read by Claude Code) or `.github/copilot-instructions.md` (auto-read by GitHub Copilot)	Loaded automatically by the tool at session start. No manual step required.
Current codebase state	Git repository	The agent inspects the repository independently. Agents MUST NOT assume the codebase matches any prior description; they verify it directly using grep, file reads, and test execution.

Outputs produced by the coding layer:

Artifact	Description	Destination
Code changes	Committed to the repository per the git conventions in §23.5.	Git repository (committed to branch or `main`)
Test results	Pass/fail status and coverage deltas. Recorded in the session report.	`.handoff/reports/` (within the session report)
Changelog entries	Additions to `CHANGELOG.md` reflecting completed work.	Repository root
Session report	A structured summary of the session's activities, results, and observations. See §23.3.4.	`.handoff/reports/`

The coding layer MUST NOT make architectural decisions, redefine module boundaries, or deviate from the specification without explicit authorization from the admin layer.

23.3. Handoff Protocol

The handoff protocol is the critical integration point between layers. All task definitions and results MUST pass through written, version-controlled artifacts. This section defines the directory structure, artifact formats, and procedures for both directions of handoff.

23.3.1. Handoff Directory Structure

```
.handoff/
├── plans/        # Admin → Coding (sprint docs, templates)
└── reports/      # Coding → Admin (session reports, test summaries)
```

Both subdirectories use the standard ShruggieTech naming convention: `<YYYYmmdd>-<ZZZ>-<title>.<ext>`, where `ZZZ` is a three-digit zero-padded increment that resets to `001` on each new date. Related artifacts (e.g., a sprint document and its paired template) share the same date and increment value.

The `.handoff/` directory is tracked in version control. All artifacts committed here become part of the project's permanent record. When a sprint is fully complete and its reports have been reviewed, the associated plan and report files MAY be moved to `.archive/` to keep `.handoff/` focused on active or recent work.

23.3.2. Admin-to-Coding Handoff

The admin layer produces the following artifacts and commits them to `.handoff/plans/`:

Sprint document. A markdown file following the five-section structure defined in the metadexer overview (Appendix D). The document contains:

1. **Header block.** Sprint identifier (date-increment format), target component, target phase, and estimated scope.
2. **Purpose and context.** What this sprint achieves and why it matters. References to specification sections that govern the work.
3. **Implementation ordering.** The sequence in which work items MUST be executed, with dependency declarations between items.
4. **Work item sections.** Each work item is a self-contained unit with: a description of the task, a list of affected files (the "affected file matrix"), specific implementation instructions, and acceptance criteria expressed as verifiable commands or assertions.
5. **Specification update directive.** Instructions for any specification changes that should accompany the code changes (if applicable).

Each work item section MUST be written so that an AI coding agent can execute it within a single context window, without interactive clarification. If a work item requires more context than fits in a single window, it MUST be decomposed into smaller items.

Session prompt template. A `_TEMPLATE.txt` file paired with the sprint document (same date and increment). This file contains the literal text to be pasted into a coding agent's initial prompt. It typically includes: a one-paragraph context summary, the path to the sprint document, the specific work item number(s) to execute, and any special instructions (e.g., "verify against PostgreSQL backend only").

Procedure:

1. The admin layer authors the sprint document and template.
2. Both files are committed to `.handoff/plans/` with the message: `workflow: add sprint <YYYYMMDD>-<ZZZ>`.
3. The project lead pushes the commit to `main` (or merges it if authored on a branch).
4. The coding layer pulls `main` and reads the sprint document to initiate work.

23.3.3. Coding-to-Admin Handoff

When a coding session concludes (whether the sprint is complete, partially complete, or blocked), the coding layer produces a session report and commits it to `.handoff/reports/`.

What gets committed:

1. **Session report** (required). A structured markdown file following the format in §23.3.4.
2. **Changelog update** (required if work was completed). Additions to `CHANGELOG.md` at the repository root.
3. **Test evidence** (required if tests were executed). Test output MAY be embedded in the session report or committed as a separate file alongside it.

Procedure:

1. The coding agent (or the project lead on the agent's behalf) authors the session report.
2. The session report is committed to `.handoff/reports/` with the message: `workflow: add session report <YYYYMMDD>-<ZZZ>`.
3. If the agent cannot generate the report itself (due to tool limitations), the project lead authors it manually based on the agent's commit history and test output. The report MUST still follow the defined format.
4. The admin layer reviews the session report, the commit history, the changelog, and the test results before initiating the next planning cycle.

23.3.4. Session Report Format

Every session report is a markdown file committed to `.handoff/reports/`. The filename follows the standard naming convention (e.g., `20260310-001-sprint-001-session-report.md`). The file MUST contain the following sections in order:

```
# Session Report: <Sprint ID>, <Work Item(s)>

- **Date:** <YYYY-MM-DD>
- **Agent:** <tool name and mode, e.g., "Claude Code interactive", "VS Code
Background Agent">
- **Sprint reference:** <path to sprint document in .handoff/plans/>
- **Work items addressed:** <comma-separated list of item numbers>
- **Status:** <Complete | Partial | Blocked>

## Changes Made

<A file-level summary of all modifications. For each file changed, state what was
done and why. This is not a diff; it is a human/AI-readable narrative of the
changes.>

## Test Results

<Pass/fail summary. Include the exact pytest invocation and its output summary
line.
If coverage was measured, include the delta. Example:>

```
$ pytest
====== 47 passed in 3.21s ======
```

Deviations from Sprint Plan
```

```
<Any cases where the agent deviated from the sprint document's instructions.
Each deviation MUST state what was specified, what was done instead, and why.
If no deviations occurred, state "None.">

## Issues and Observations

<Anything the admin layer should be aware of: bugs discovered, ambiguities in the
specification, performance concerns, suggestions for future work. If none, state
"None.">
```

The session report is the primary mechanism by which the admin layer gains visibility into what happened during a coding session. It MUST be accurate and complete enough that the admin layer can make informed planning decisions without re-reading every commit diff.

23.3.5. Agent Session Transcript Preservation

Agent session transcripts (the full conversation log between a human or orchestrator and the AI coding agent) are a valuable debugging and auditing resource. However, transcript export mechanisms vary by tool:

Tool	Transcript access
Claude Code (interactive)	Conversation history is accessible via the Claude Code CLI. Export by copying the session log or using built-in export commands if available.
VS Code Background Agent	Session logs are accessible via the Chat panel history. Content can be copied from the panel.
VS Code Cloud Agent	Session logs are accessible via the GitHub Copilot interface.
Superset	Per-agent transcripts are accessible via the Superset task view.

Transcripts are NOT committed to the repository by default (they are typically too large and contain tool-specific formatting). Instead, the project lead preserves them locally when needed for debugging. The session report (§23.3.4) serves as the lightweight, version-controlled summary of what occurred.

When a transcript is needed for debugging agent behavior, the project lead extracts the relevant portions and includes them in the session report's "Issues and Observations" section, or files them as a separate document in `.handoff/reports/` with a descriptive filename (e.g., `20260310-002-transcript-excerpt-vault-bug.md`).

23.4. Agent Execution Model

The coding layer supports three execution modes. The project lead selects the appropriate mode based on task characteristics.

23.4.1. Sequential Mode

A single agent session processes one work item at a time. The agent operates directly on the main branch. Changes are reviewed and committed before the next work item begins.

When to use: Simple, low-risk changes. Bug fixes with narrow scope. Specification updates. Tasks where the overhead of worktree setup exceeds the time saved by parallelism.

Procedure:

1. Open a coding agent session (VS Code local agent, Claude Code interactive, or equivalent).
2. Provide the sprint document and template as context.
3. The agent implements the work item.
4. Review the changes. Run acceptance criteria verification commands from the sprint document.
5. Commit to main. Proceed to the next work item.

23.4.2. Parallel Independent Mode

Multiple agent sessions run simultaneously, each in an isolated Git worktree on a separate branch. Agents work on independent tasks and cannot interfere with each other.

When to use: Two or more work items from the same sprint are independent (no shared file modifications, no sequential dependency). Tasks are well-defined and self-contained. Time savings from parallelism justify the merge overhead.

Procedure:

1. Identify independent work items in the sprint document. Verify independence by checking the "affected file matrix" in each work item section. If two items modify the same file, they are NOT independent and MUST be executed sequentially.
2. For each independent work item, start a separate agent session in its own worktree:
   - **Using VS Code:** Start a Background Agent session per task (Chat view > New Chat (+) > New Background Agent > select "Worktree" isolation).
   - **Using Superset:** Create a new task (Cmd+T) per work item. Superset handles worktree creation automatically.
   - **Using Claude Code directly:** Use the `--worktree` flag or `isolation: worktree` in the agent definition.
3. Provide each agent with: the relevant work item section from the sprint document, the specification, and the `CLAUDE.md` context file.
4. Monitor agent progress. Intervene only if an agent requests clarification or stalls.
5. When an agent completes, review the diff against the sprint document's acceptance criteria. Run the specified verification commands.
6. Merge completed branches into main one at a time, in the dependency order specified by the sprint document's implementation ordering section:

   ```
   git checkout main
   git merge --no-ff <branch-name>
   ```

   If using VS Code background agents, use the "Apply" action in the Chat view instead of manual merge commands.
7. After each merge, verify that the cumulative codebase passes all tests before proceeding to the next merge.

8. Delete completed worktrees after successful merge.

Branch naming convention:

```
sprint/<YYYYMMDD>-<ZZZ>/<work-item-number>-<short-description>
```

Example: `sprint/20260310-001/03-vault-put-operation`

23.4.3. Coordinated Team Mode

A lead agent decomposes a complex task and delegates subtasks to multiple subordinate agents that can communicate with each other. This mode is experimental and carries significantly higher token costs.

When to use: Complex tasks that benefit from multiple perspectives operating on the same logical problem (e.g., parallel exploration of architecture options, simultaneous implementation and test writing for a tightly coupled module). Reserved for tasks where the cost is justified by the complexity.

Procedure (Claude Code Agent Teams):

1. Verify the `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` environment variable is set to `1`.
2. Start a Claude Code session and describe the task, specifying how many teammates to spawn and what each should focus on.
3. The lead agent coordinates work through a shared task list. Monitor progress in split-pane terminal mode (requires tmux or iTerm2).
4. Review the combined output. The lead agent synthesizes results from all teammates.
5. Commit the coordinated result to main (or to a worktree branch for review first).

Cost expectation: Approximately 5x the token usage of a single-agent session per spawned teammate. Use sparingly and only for tasks where the coordination overhead produces measurably better outcomes.

23.5. Git Conventions

23.5.1. Branch Strategy

The project uses a single `main` branch as the integration target. All work, whether executed sequentially or in parallel, merges to `main`.

Feature branches exist only as temporary worktree branches during parallel execution. They are deleted after merge. Long-lived feature branches are not used.

23.5.2. Commit Discipline

- Each logical change is one commit. Agents SHOULD NOT bundle unrelated changes into a single commit.
- Commit messages follow the pattern: `<module>: <imperative description>` (e.g., `vault: implement put operation for local backend`).
- Sprint-related commits MAY reference the sprint document in the message body: `Sprint: 20260310-001, Item 3`.

- Merge commits from parallel branches use `--no-ff` to preserve branch history in the log.

23.5.3. Worktree Cleanup

After a successful merge, the worktree and its branch MUST be deleted:

```
git worktree remove <path>
git branch -d <branch-name>
```

If using Superset or VS Code, worktree cleanup is handled automatically when you dismiss a completed task or delete an agent session.

23.6. Agent Context Management

23.6.1. Agent Context Files

The repository maintains two agent context files that provide persistent project-level context to AI coding agents. Both files are the admin layer's responsibility to maintain and keep synchronized.

`CLAUDE.md` at the repository root. Read automatically by Claude Code at session start. This is the primary agent context file.

`.github/copilot-instructions.md` in the `.github/` directory. Read automatically by GitHub Copilot in VS Code. This is the secondary agent context file.

Required contents (identical in both files):

- Project identity: name (`metadexer`), organization (`ShruggieTech LLC`), language (`Python 3.12+`).
- Repository structure summary: top-level directory layout with one-line descriptions.
- Specification location and authority hierarchy: `metadexer-spec.md` is authoritative for metadexer; `shruggie-indexer-spec.md` (in the indexer repo) is authoritative for the indexer. The specification overrides all other documents when they conflict.
- Coding conventions: UTF-8 without BOM, LF line endings, `snake_case` for Python identifiers, `import` ordering (`stdlib` / `third-party` / `local`), standard `logging` module usage.
- Testing conventions: `pytest` invocation, `--strict-markers`, test directory structure by type (§17).
- CLI conventions: `click` for argument parsing, `stdout` for structured output, `stderr` for diagnostics, `--dry-run` for side-effecting operations.
- Explicit prohibitions: no silent data loss, no implicit deletion, no recomputing identity, no architectural decisions without admin layer authorization.

Tool-specific preamble (differs between files):

Each file MAY include a brief preamble at the top with instructions specific to the tool that reads it. For example, `CLAUDE.md` may include Claude Code-specific conventions for tool use or file handling. `.github/copilot-instructions.md` may include Copilot-specific conventions for suggestion behavior. The preamble MUST NOT contradict the shared core content.

Synchronization rule: When the admin layer updates one file, it MUST update the other in the same commit. The commit message for agent context updates is: `docs: update agent context files`.

23.6.2. Sprint Document as Agent Context

When executing a work item, the coding agent's context MUST include:

1. The agent context file (`CLAUDE.md` or equivalent) -- always.
2. The specific work item section from the sprint document -- always.
3. The authoritative specification for the component being modified -- always.
4. The `_TEMPLATE.txt` prompt file, if one is paired with the sprint document -- when available.
5. Any referenced prior sprint documents or changelogs -- only when the work item explicitly depends on prior work.

Agents MUST NOT be given the entire sprint document if they are only executing one work item. Unnecessary context degrades agent performance and increases cost.

23.6.3. Agent Session Discipline

- Agents MUST NOT trust prior implementation work without independent verification.
- Agents MUST use grep-based evidence collection to verify codebase state before making changes.
- Agents MUST verify acceptance criteria against actual runtime behavior, not assumed correctness from code inspection alone.
- Each work item section is self-contained. An agent implementing Item 3 MUST NOT assume that Item 2 has been completed unless the sprint document's implementation ordering explicitly establishes that dependency and the agent has verified the dependency is satisfied in the current codebase.

23.7. Review and Integration

23.7.1. Review Checklist

Before merging any agent-produced changes (whether from sequential or parallel execution), the project lead verifies:

1. **Acceptance criteria:** Every criterion listed in the sprint document's work item is met, verified by running the specified commands.
2. **Specification compliance:** Changes conform to the authoritative specification. No new behavior is introduced that the specification does not define or permit.
3. **Test passage:** `pytest` runs clean with no failures or unexpected warnings.
4. **Encoding:** All new or modified files use UTF-8 without BOM and LF line endings. Spot-check with `file --mime-encoding <path>`.
5. **No scope creep:** Changes are limited to what the sprint document authorized. Agents occasionally introduce "helpful" improvements outside the defined scope; these are reverted or deferred to a future sprint.
6. **Changelog:** `CHANGELOG.md` reflects the changes accurately.

23.7.2. Merge Conflict Resolution

When merging parallel branches, conflicts may arise if:

- Two agents modified the same file despite the affected file matrix indicating independence (an error in the sprint document).

- An agent modified a shared dependency (e.g., `__init__.py` imports, `pyproject.toml` dependencies).

Conflict resolution is the project lead's responsibility. Resolution follows these rules:

- If both changes are correct and non-overlapping within the file, combine them manually.
- If the changes are logically conflicting (two different approaches to the same problem), choose one and file a follow-up item for the discarded approach if needed.
- After resolving conflicts, re-run the full test suite to verify the combined result.

23.8. Tooling Configuration

23.8.1. Recommended Stack

Layer	Primary Tool	Alternative
Admin	Claude.ai (Project-based)	Any browser-based AI chat with persistent context
Coding (interactive)	VS Code Local Agent or Claude Code (interactive)	Any IDE with integrated AI chat
Coding (parallel)	Superset + Claude Code	VS Code Background Agents
Coding (cloud/CI)	VS Code Cloud Agent (Copilot coding agent)	Claude Code on web (claude.ai/code)
Review	VS Code (final codebase), Superset (per-agent diffs)	`git diff` on the command line

23.8.2. VS Code Configuration

The following VS Code settings are recommended for multi-agent workflows:

```
{
  "chat.agent.enabled": true,
  "github.copilot.chat.claudeAgent.enabled": true,
  "editor.formatOnSave": true,
  "files.encoding": "utf8",
  "files.eol": "\n"
}
```

23.8.3. Superset Configuration

If Superset is used for orchestration, the repository SHOULD include a `.superset/` directory with:

```
.superset/
├── config.json
├── setup.sh
└── teardown.sh
```

config.json:

```
{
  "setup": ["./.superset/setup.sh"],
  "teardown": ["./.superset/teardown.sh"]
}
```

setup.sh:

```
#!/bin/bash
# Copy environment variables from the root worktree
cp ../.env .env 2>/dev/null || true
# Install development dependencies
pip install -e ".[dev]" --quiet
echo "Worktree ready."
```

teardown.sh:

```
#!/bin/bash
echo "Worktree cleanup complete."
```

23.9. Workflow Summary

The following sequence describes one complete development cycle from planning through integration.

1. **Admin layer** reviews the current state of the project (recent commits, test results, session reports in `.handoff/reports/`).
2. **Admin layer** authors a sprint document defining the next batch of work items. The document follows the five-section structure.
3. **Admin layer** commits the sprint document and its `_TEMPLATE.txt` to `.handoff/plans/`.
4. **Project lead** reviews the sprint document's work items and determines the execution mode:
   - If all items are sequential dependencies: use Sequential Mode (§23.4.1).
   - If some items are independent: use Parallel Independent Mode (§23.4.2) for independent items, Sequential Mode for dependent items.
   - If a single complex item benefits from multi-perspective exploration: use Coordinated Team Mode (§23.4.3).
5. **Coding layer** executes work items per the selected mode.
6. **Coding layer** produces session report(s) and commits them to `.handoff/reports/`.
7. **Project lead** reviews agent output per the Review Checklist (§23.7.1).
8. **Coding layer** merges approved changes to main.
9. **Project lead** verifies the cumulative codebase state (full test suite, encoding checks).
10. **Admin layer** consumes the coding layer's output (session reports, commits, changelog, test results) and begins planning the next cycle.

24. Composition Rules

No module MAY:

- Recompute identity unless performing an explicit integrity verification.
- Rewrite or amend an IndexEntry outside of the indexer's own operation.
- Implicitly delete content from any backend.
- Implicitly migrate content between backends.

Cross-module interaction within metadexer occurs through defined internal APIs. The indexer communicates with metadexer exclusively through the IndexEntry JSON contract.

25. Future Considerations

The following capabilities are achievable through future extension without requiring changes to the identity model or core contracts:

- Multiple vault backends operating simultaneously.
- Multiple catalog instances (e.g., per-tenant, per-project).
- Vault-to-vault migration with integrity verification.
- Manifest export and import for portable asset sets.
- Snapshot materialization from catalog state.
- Immutable archival tiers with tiered access policies.
- A web-based UI layer for search and browsing (API-driven, thin over CLI logic).
- Integration with hotwire for automated real-time feed ingestion.
- Embedding and vector search extensions for semantic retrieval.

None of these capabilities require redefining content identity. The IndexEntry contract is the stable foundation on which all future extension is built.

Document History

Date	Version	Change
2026-03-07	DRAFT	Initial specification. Derived from the metadexer high-level overview (`20260305-004-metadexer-overview.md`). Establishes architectural contracts, module responsibilities, invariants, and development phasing sufficient for sprint planning.

Date	Version	Change
2026-03-09	DRAFT	Merged the standalone development workflow document (`metadexer-development-workflow.md`, dated 2026-03-08) into the specification as §23. Expanded handoff protocol with concrete artifact formats, session report schema, and agent session transcript preservation guidance. Introduced `.handoff/plans/` and `.handoff/reports/` directory structure, replacing the prior use of `.archive/` as the handoff location. `.archive/` is retained for historical document storage only. Added `CLAUDE.md` and `.github/copilot-instructions.md` as defined agent context files with synchronization requirements. Updated repository structure (§10) to reflect new directories and files. Added workflow-related terms to the terminology table (§1.5). Renumbered §23 (Composition Rules) to §24 and §24 (Future Considerations) to §25.